

Universidad Mariano Gálvez de Guatemala
Sede Sololá
Facultad de Ingeniería en Sistemas de Información
Base de Datos I
Ing. Wilmer Xeron



UNIVERSIDAD
MARIANO GÁLVEZ

CrediCore Fase I: Cimientos de Titanio (DDL y Dominios)
Cristhian Cesar Felipe Mucún Queché
2290-17-17851

Introducción:

En esta documentación presento la primera fase de CREDICORE, que corresponde a la creación de la estructura de la base de datos utilizando lenguaje DDL en SQL Server, trabajado desde DBeaver conectado a la instancia de SQL Server que tenemos ejecutándose mediante contenedores.

En esta fase se trabajaron principalmente esquemas, tipos de datos, llaves primaria, IDENTITY, y restricciones como UNIQUE, DEFAULT y CHECK.

1. Creación de la Base de Datos y su uso:

En esta parte utilizamos dos comandos, uno que es el encargado de crear la base de datos que posteriormente usaremos para crear nuestras tablas.

Descripción	Valor usado	Por qué
Creación de BD	CREATE DATABASE CrediCoreDB;	Creamos la base de datos y le asignamos un nombre.
Uso de la BD	USE CrediCoreDB;	Le indicamos que base de datos usaremos, para la creación de tablas

2. Creación de esquemas:

Creamos dos esquemas que serán nuestros contenedores lógicos, para organizar nuestras tablas y controlar mejor la seguridad y los permisos de acceso.

Descripción	Valor usado	Por qué
Esquema Operaciones	CREATE SCHEMA Operaciones;	Donde se almacenará la tabla Clientes y Créditos.
Esquema Garantía	CREATE SCHEMA Garantia;	Donde almacenaremos la tabla Vehiculos

3. Creación de la primera tabla:

Creamos la primera tabla que contendrá a nuestros clientes, el cual contendrá las columnas (IdCliente, Nombre, Apellido, DPI, Telefono, Correo).

** Con una restricción la cual es:

- El campo DPI debe tener restricción UNIQUE que no permitirá registrar el mismo cliente dos veces.

Valor usado	Por qué
CREATE TABLE Operaciones.Clientes(	Creación y asignación de nombre a la tabla.
IdCliente INT IDENTITY(1,1) NOT NULL,	Creación del identificador del cliente, el cual con el IDENTITY estará incrementando de 1 en 1 y no permitirá datos nulos o vacíos.
Nombre VARCHAR(50) NOT NULL,	Creación de la columna Nombre con un VARCHAR que permitirá 50 caracteres y no permitirá datos nulos o vacíos.
Apellido VARCAHR(50) NOT NULL,	Creación de la columna Apellido con un VARCHAR que permitirá 50 caracteres y no permitirá datos nulos o vacíos.

DPI CHAR(18) NOT NULL,	Creación de la columna DPI con un CHAR que permitirá 18 caracteres y no permitirá datos nulos o vacíos.
Telefono VARCHAR(25) NOT NULL,	Creación de la columna Teléfono con un VARCHAR que permitirá 25 caracteres y no permitirá datos nulos o vacíos.
Correo VARCHAR(50) NOT NULL,	Creación de la columna Correo con un VARCHAR que permitirá 50 caracteres y no permitirá datos nulos o vacíos.
CONSTRAINT PK_Clientes_IdCliente PRIMARY KEY (IdCliente),	Asignamos una regla en la cual indicamos que nuestra llave primaria será IdCliente y tendrá un nombre fijo que será PK_Clientes_IdCliente.
CONSTRAINT UQ_Clientes_DPI UNIQUE (DPI));	Asignamos la regla inquebrantable para que no permita ingresar dos clientes con el mismo DPI.

En esta etapa ya cumplimos con la restricción de que no se permita ingresar dos clientes con el mismo DPI, y le asignamos que nuestra llave primaria será IdCliente. Una de las diferencias a destacar es que, en Nombre, Apellido usamos VARCHAR pero en DPI CHAR, ¿Por qué?

Esto es algo simple que encontramos en la documentación oficial de SQL, la diferencia es que ***VARCHAR tiene una longitud variable y solo usa el espacio necesario para el texto real, ahora lo que es CHAR tiene una longitud fija y siempre usa el mismo espacio de disco, por eso en DPI escogemos el dato CHAR ya que es un dato fijo, en cambio en nombres y apellidos son datos variables.***

4. Creación de la Segunda tabla:

Creamos la segunda tabla que contendrá los vehículos, el cual contendrá las columnas (IdVehiculo, Modelo, Marca, NumeroPlaca, NumeroChasis, AnioFabricacion, Color, NumeroTituloPropiedad).

** Con las restricciones las cuales son:

- El campo AnioFabricacion tiene prohibido aceptar vehículos con más de 15 años de antigüedad es decir que sean mayores al año 2011 como garantía.
- Los campos NumeroPlaca, NumeroChasis deben ser únicas y no se deben aceptar los mismos números.

Valor usado	Por qué
CREATE TABLE Garantia.Vehiculos(	Creación y asignación de nombre a la tabla.
IdVehiculos INT IDENTITY(1,1) NOT NULL,	Creación del identificador del vehiculos, el cual con el IDENTITY estará incrementando de 1 en 1 y no permitirá datos nulos o vacíos.
Modelo VARCHAR(50) NOT NULL,	Creación de la columna Modelo con un VARCHAR que permitirá 50 caracteres y no permitirá datos nulos o vacíos.

Marca VARCHAR(80) NOT NULL,	Creación de la columna Modelo con un VARCHAR que permitirá 80 caracteres y no permitirá datos nulos o vacíos.
NumeroPlaca VARCHAR(25) NOT NULL,	Creación de la columna NumeroPlaca con un VARCHAR que permitirá 25 caracteres y no permitirá datos nulos o vacíos.
NumeroChasis VARCHAR(50) NOT NULL,	Creación de la columna NumeroChasis con un VARCHAR que permitirá 50 caracteres y no permitirá datos nulos o vacíos.
AnioFabricacion SMALLINT NOT NULL,	Creación de la columna AnioFabricacion con un tipo de dato SMALLINT que no permitirá datos nulos o vacíos.
Color VARCHAR(40) NULL,	Creación de la columna Color con un VARCHAR que permitirá 40 caracteres y permitirá datos nulos o vacíos.
NumeroTituloPropiedad VARCHAR(80) NOT NULL,	Creación de la columna NumeroTituloPropiedad con un VARCHAR que permitirá 80 caracteres y no permitirá datos nulos o vacíos.
CONSTRAINT PK_Vehiculos_IdVehiculo PRIMARY KEY (IdVehiculo);	Asignamos una restricción en el cual indicamos que nuestra llave primaria será IdVehiculos y tendrá un nombre fijo PK_Vehiculos_IdVehiculo.
CONSTRAINT CK_Vehiculos_AnioFabricacion CHECK (AnioFabricacion >= 2011),	Asignamos una restricción en el cual AnioFabricacion no aceptara vehículos menores al año 2011 ya que la garantía no acepta vehículos con más de 15 años de antigüedad.
CONSTRAINT UQ_Vehiculos_NumeroPlaca UNIQUE (NumeroPlaca),	Asignamos la restricción de que el NumeroPlaca será único, quiere decir que no aceptara vehículos con el mismo número de placa.
CONSTRAINT UQ_Vehiculos_NumeroChasis UNIQUE (NumeroChasis));	Le asignamos también al NumeroChasis que sea único y que no acepte vehículos con el mismo número de chasis.

Ojo acá usamos un nuevo tipo de datos que es el SMALLINT, ¿Por qué? Y es porque ocupa un espacio de 2 bytes y para años está muy bien usar este tipo de datos porque ayuda a ahorrar memoria. Otra de las cosas es que en columnas donde van números usamos VARCHAR y esto es porque INT lo usamos únicamente para datos que contendrán operaciones matemáticas como $2 + 2$, en el caso de NumeroPlaca no la estaremos operando matemáticamente entonces no es necesario.

5. Creación de la Tercera tabla:

Creamos la tercera tabla y última para esta fase uno, la cual contendrá los Créditos y las columnas (IdCreditos, IdCliente, IdVehiculo, MontoCapitalOtorgado, TasaInteresMensual, Estado, FechaDesembolso).

** Con las restricciones que son:

- La tasa de interés no puede ser negativa.
- El monto otorgado debe ser mayor a Q1,000.00

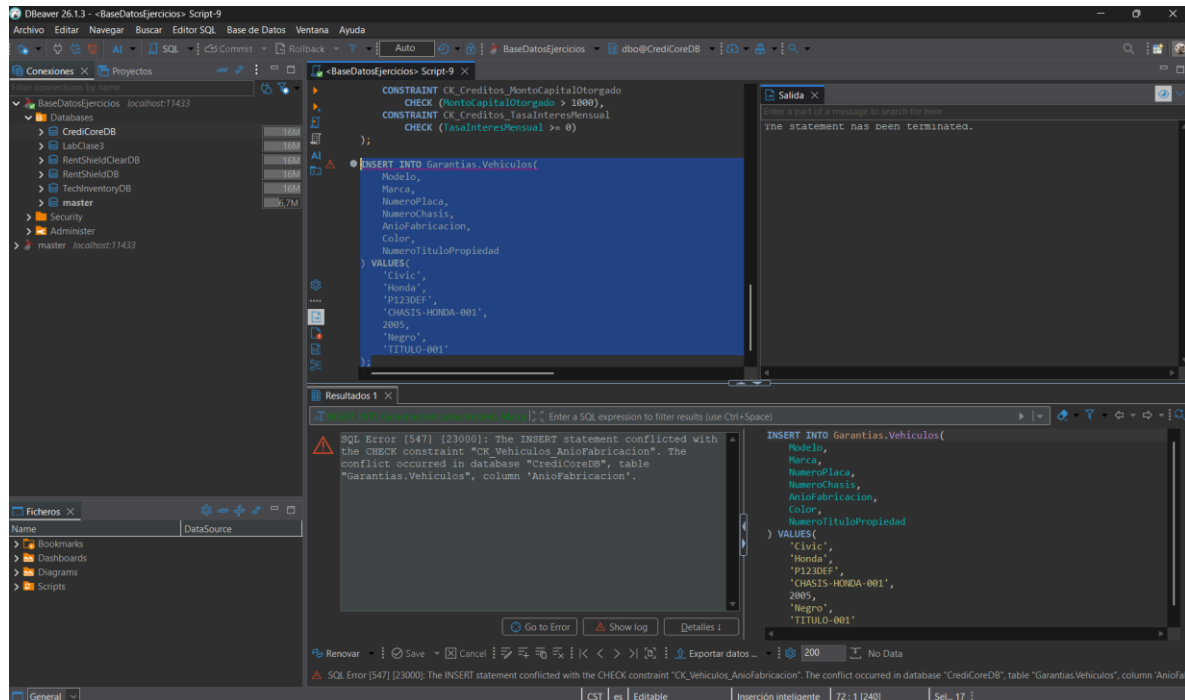
- El campo estado por defecto tiene que ser 'Activo'.
- La fecha debe capturar la fecha actual del servidor.

Valor usado	Por qué
CREATE TABLE Operaciones.Creditos(	Creación y asignación de nombre a la tabla.
IdCredito INT IDENTITY(1,1) NOT NULL,	Creación del identificador del credito, el cual con el IDENTITY estará incrementando de 1 en 1 y no permitirá datos nulos o vacíos.
IdCliente INT NOT NULL,	Creación de la columna IdCliente de tipo INT el cual posteriormente lo relacionaremos con la tabla clientes, por el momento solo no acepta datos nulos o vacíos.
IdVehiculos INT NOT NULL	Creación de la columna IdVehiculos de tipo INT el cual posteriormente lo relacionaremos con la tabla vehiculos, por el momento solo no acepta datos nulos o vacíos.
MontoCapitalOtorgado DECIMAL(18,2) NOT NULL,	Creación de la columna MontoCapitalOtorgado con un dato DECIMAL que permitirá 18 dígitos, 2 decimales y no permitirá datos nulos o vacíos.
TasaInteresMensual DECIMAL(5,2) NOT NULL,	Creación de la columna TasaInteresMensual con un dato DECIMAL que permitirá 5 dígitos, 2 decimales y no permitirá datos nulos o vacíos.
Estado VARCHAR(20) NOT NULL CONSTRAINT DF_Creditos_Estado DEFAULT ('Activo'),	Creación de la columna Estado con VARCHAR que permitirá 20 caracteres y no permitirá datos nulos o vacíos, con la restricción que por defecto tendrá 'Activo' por defecto.
FechaDesembolso DATETIME NOT NULL CONSTRAINT DF_Creditos_FechaDesembolso DEFAULT (GETDATE()),	Creación de la columna FechaDesembolso con DATETIME que no permitirá datos nulos o vacíos, con la restricción que por defecto tendrá la fecha del servidor
CONSTRAINT CK_Creditos_MontoCapitalOtorgado CHECK (MontoCapitalOtorgado > 1000),	Asignamos la restricción que MontoCapitalOtorgado no puede ser menor que Q1,000.00
CONSTRAINT CK_Creditos_TasaInteresMensual CHECK (TasaInteresMensual >= 0));	Asignamos la restricción que TasaInteresMensual que el interés no puede ser negativo.

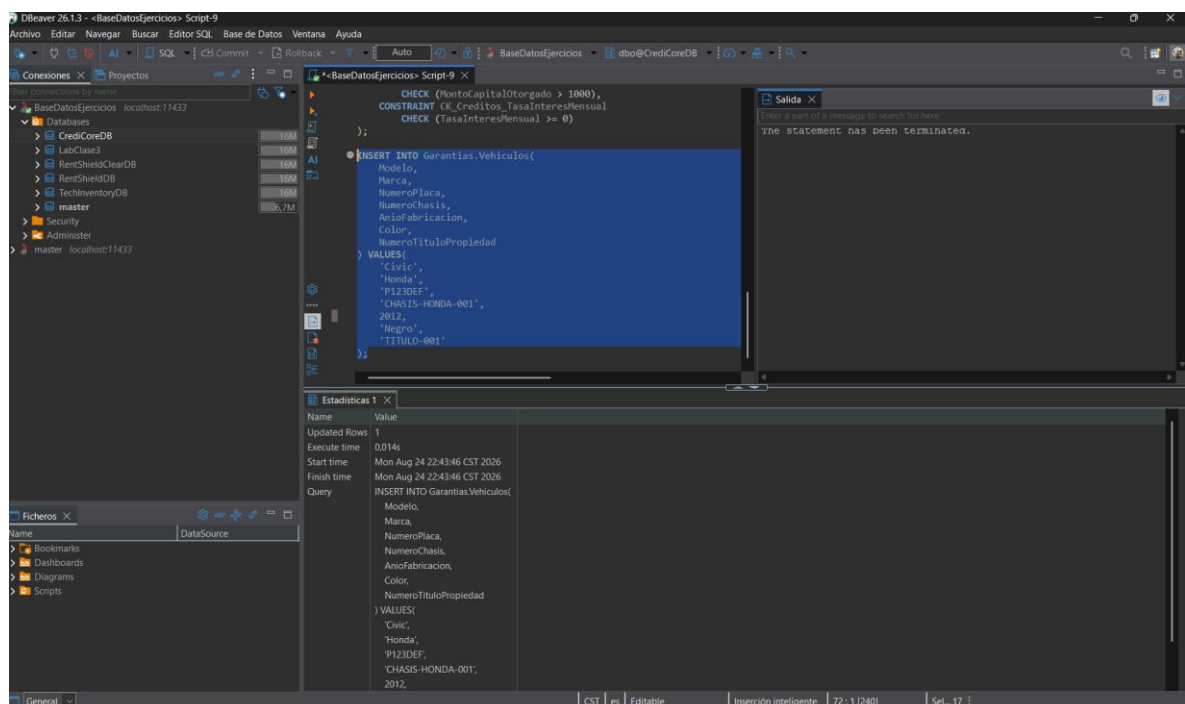
Ojo acá usamos nuevos tipos de datos que son DECIMAL, DATETIME, una de las grandes diferencias que tenemos que entender es porque usamos DECIMAL y no INT, es porque INT guarda datos enteros es decir 1000.00 exactos, ahora DECIMAL guarda números con partes fraccionarias que esto nos funciona muy bien para lo que queremos hacer que es guardad el monto capital otorgado que por la tasa de interés puede variar y tener un número 2385.25 y no usamos FLOAT por el tema que hace aproximaciones binarias.

Que es DATETIME es un tipo de dato que sirve para guardar fecha y hora juntas en una misma columna el cual nos ayuda a guardar la fecha y hora que recuperamos del servidor.

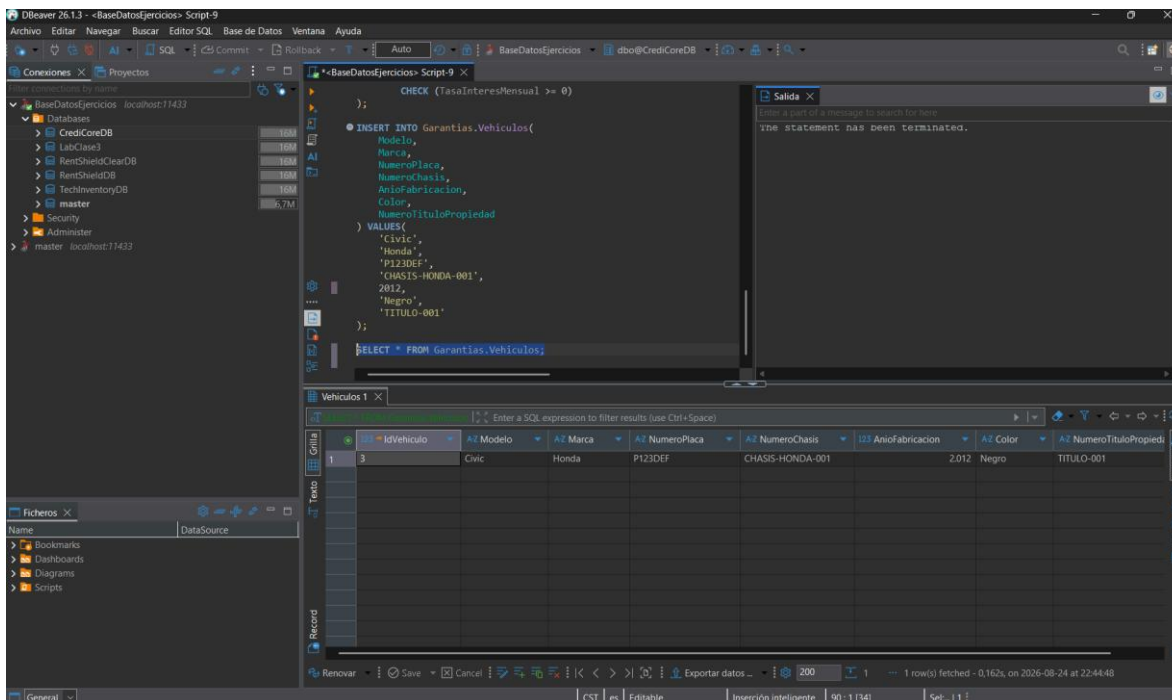
6. Prueba de destrucción:



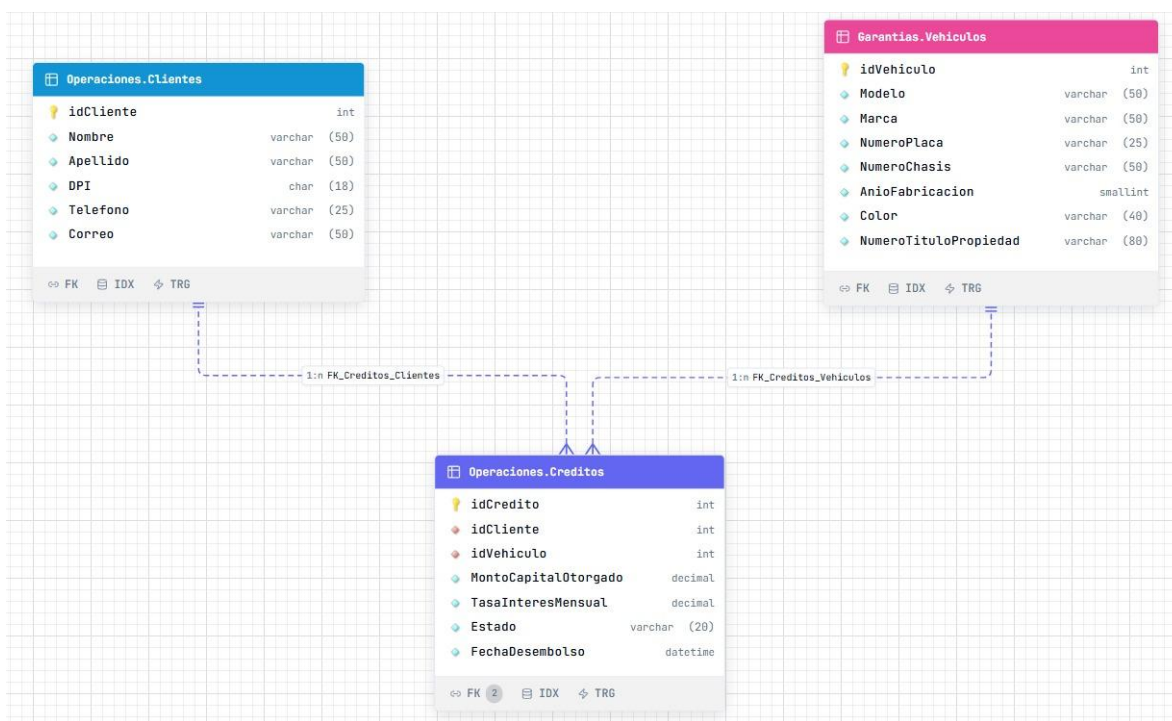
En este caso intentamos ingresar un año de fabricación invalido, como la restricción decía que solo se permiten vehículos con 15 años de antigüedad, si vemos el año ingresado es del 2005 es decir 21 años de antigüedad y bloquea perfectamente el ingreso de ese dato.



Pero veamos acá que el registro si pasa porque el año es 2012 que son 14 años de antigüedad por lo que el registro es correcto.



7. Diagrama conceptual de cómo visualizar la conexión de las tablas en un futuro:



El diagrama conceptual representa una posible evolución futura de CrediCore. Se plantea una relación de uno a muchos entre clientes y créditos, ya que un cliente puede adquirir distintos créditos a lo largo del tiempo, mientras que cada crédito corresponde a un único cliente. De igual forma, se visualiza una relación de uno a muchos entre vehículos y créditos, permitiendo que un vehículo pueda respaldar diferentes operaciones históricas. Se implementará posteriormente mediante llaves foraneas

8. GitHub, Youtube:

github.com/Cristhian1997-dev/CrediCoreDB.git

<https://www.youtube.com/watch?v=nzKtoZ8DUGc>